

C++ Synchronized Buffered Ostream

ISO/IEC JTC1 SC22 WG21 p0053r2 - 2016-07-05

Lawrence Crowl, Lawrence@Crowl.org

Peter Sommerlad, Peter.Sommerlad@hsr.ch

Nicolai Josuttis

[Introduction](#)

[Solution](#)

[Design](#)

[Feature Test](#)

[Wording](#)

[17.6.1.2 Headers \[headers\]](#)

[27.1 General \[input.output.general\]](#)

[27.3 Forward declarations \[iostream.forward\]](#)

[27.12 Synchronized output stream \[syncstream\]](#)

[27.12.1 Overview \[syncstream.overview\]](#)

[27.12.2 Class template basic_osyncstream \[syncstream.osyncstream\]](#)

[27.12.2.1 Constructor \[syncstream.osyncstream.ctor\]](#)

[27.12.2.2 Destructor \[syncstream.osyncstream.dtor\]](#)

[27.12.2.3 Assignment \[syncstream.osyncstream.assign\]](#)

[27.12.2.4 Member Functions \[syncstream.osyncstream.mfun\]](#)

[Implementation](#)

[Revisions](#)

[References](#)

Introduction

At present, some stream output operations guarantee that they will not produce race conditions, but do not guarantee that the effect will be sensible. Some form of external synchronization is required. Unfortunately, without a standard mechanism for synchronizing, independently developed software will be unable to synchronize.

[N3535 C++ Stream Mutexes](#) proposed a standard mechanism for finding and sharing a mutex on streams. At the Spring 2013 standards meeting, the Concurrency Study Group requested a change away from a full mutex definition to a definition that also enabled buffering.

[N3678 C++ Stream Guards](#) proposed a standard mechanism for batching operations on a stream. That batching may be implemented as mutexes, as buffering, or some combination of both. It was the response to the Concurrency Study Group. A draft of that paper was reviewed in the Library Working Group, who found too many open issues on what was reasonably exposed to the 'buffering' part.

[N3665 Uninterleaved String Output Streaming](#) proposed making streaming of strings of length less than `BUFSIZ` appear uninterleaved in the output. It was a "minimal" functionality change to the existing standard to address the problem. The full Committee chose not to adopt that minimal solution.

[N3750 C++ Ostream Buffers](#) proposed an explicit buffering, at the direction of the general consensus in the July 2013 meeting of the Concurrency Study Group. In November 2014 a further updated version [N4187](#) was discussed in Library Evolution in Urbana and it was consensus to work with a library expert to get the naming and wording better suited to LWG expectations. Nico Josuttis volunteered to help the original authors. More information on the discussion is available at [LEWG wiki](#) and the corresponding [LEWG bug tracker entry \(20\)](#). This paper address issues raised with [N4187](#). This paper has a change of title to reflect a change in class name, but contains the same fundamental approach.

Solution

We propose a `basic_osyncstream`, that buffers output operations for a wrapped stream. The `basic_osyncstream` will atomically transfer the contents of an internal stream buffer to a `basic_ostream`'s stream buffer on destruction of the `basic_osyncstream`.

The transfer on destruction simplifies the code and ensures at least some output in the presence of an exception.

The intent is that the `basic_osyncstream` is an automatic-duration variable with a relatively small scope which constructs the text to appear uninterleaved. For example,

```
{
    ostream bout(cout);
    bout << "Hello, " << "World!" << endl;
}
```

Design

While it has been pointed out that the implementation should actually in the used stream buffer object, I believe the specification should hide that effect, because such a stream buffer is not intended to be ever be used stand alone. Therefore, the specification here only provides the `basic_ostream` class template and not any details on the underlying stream buffer implementing the mechanics. I have an implementation, where the `basic_ostream` is more or less just a shell to the real meat in the stream buffer but that stream buffer is not meant to be exchanged or shared or anything.

We follow typical stream conventions of `basic_` prefixes and typedefs.

The constructor for `ostream` takes a non-const reference to a `basic_ostream` obtaining its stream buffer or a `basic_streambuf`. This stream buffer indicates that the destruction of the `ostream` may write to the stream buffer obtained with the constructor argument.

The wording below permits implementation of `basic_ostream` with either a `stream_mutex` from [N3535](#) or with implementations suitable for [N3665](#), e.g. with Posix file locks [\[PSL\]](#)

Feature Test

No header called `<ostream>` exists; testing for this header's existence is thus sufficient for testing existence of the feature.

Wording

This wording is relative to N4594.

17.6.1.2 Headers [headers]

Add a new entry to table 14 — C++ library headers :

<code><ostream></code>

27.1 General [input.output.general]

Add a new row to table 129 — Input/output library summary.

27.12 Synchronized output streams <code><ostream></code>
--

27.3 Forward declarations [iostream.forward]

Add the following forward declarations to the synopsis of `<iosfwd>` in the namespace `std`.

<pre>template <class charT, class traits = char_traits<charT>, class Allocator = allocator<charT> > class basic_ostream; typedef basic_ostream<char> ostream; typedef basic_ostream<wchar_t> wostream;</pre>
--

27.12 Synchronized output stream [ostream]

Add a new section with the following subsections.

27.12.1 Overview [ostream.overview]

<u>The header <code><ostream></code> provides a mechanism to synchronize execution agents writing to the same stream. It defines a class template <code>basic_ostream</code> to buffer output and transfer the buffered content into an object of type</u>

basic_streambuf<charT, traits> atomically with respect to such transfers by other basic_osyncstream<charT, traits, Allocator> objects referring to the same basic_streambuf<charT, traits> object. The transfer occurs when emit() is called and when the basic_osyncstream<charT, traits, Allocator> object is destroyed.

Add a synopsis for header <syncstream>.

```
template <class charT,  
          class traits,  
          class Allocator>  
class basic_osyncstream;
```

27.12.2 Class template basic_osyncstream [syncstream.osyncstream]

```
template <class charT,  
          class traits,  
          class Allocator>  
class basic_osyncstream  
: public basic_ostream<charT, traits>  
{  
public:  
    typedef charT          char_type;  
    typedef traits          traits_type;  
    typedef typename traits::int_type int_type;  
    typedef typename traits::pos_type pos_type;  
    typedef typename traits::off_type off_type;  
    typedef Allocator        allocator_type;  
    typedef basic_streambuf<charT, traits> streambuf_type;  
  
    explicit basic_osyncstream(streambuf_type *obuf, Allocator const &allocator=Allocator());  
    explicit basic_osyncstream(basic_ostream<charT, traits> &os, Allocator const &allocator=Allocator());  
    basic_osyncstream(basic_osyncstream&&);  
    ~basic_osyncstream();  
    basic_osyncstream& operator=(basic_osyncstream&&);  
    void emit();  
    streambuf_type* rdbuf_wrapped() const noexcept;  
private:  
    streambuf_type *out; // exposition only  
};
```

Allocator shall meet the allocator requirements [allocator.requirements].

[Example: Use a named variable within a block statement for streaming across multiple statements.

```
{  
    osyncstream bout(cout);  
    bout << "Hello, ";  
    bout << "World!";  
    bout << endl;  
}
```

—end example]

[Example: Use a temporary object for streaming within a single statement.

```
osyncstream(cout) << "Hello, " << "World!" << endl;
```

—end example]

27.12.2.1 Constructor [syncstream.osyncstream.ctor]

```
explicit basic_osyncstream(streambuf_type *buf, Allocator const &allocator=Allocator());
```

Effects: Stores the stream buffer pointer buf which will be the final destination of characters in out. If the stored stream buffer pointer out == nullptr output operations on *this fail. A copy of allocator is used to allocate memory for the streams own internal buffer, if any.

Constructs a stream buffer object [stream.buffer].

[Note: May construct a mutex. —end note]

[Note: If wrapped stream buffer pointer refers to a user provided stream buffer then its implementation must be aware that its member functions might be called from emit() while a lock is held. —end note]

Postconditions: out == buf

```
explicit basic_ostream(basic_ostream<charT,traits>& os, Allocator const &allocator=Allocator());
```

Effects: basic_ostream(os.rdbuf(),a)

```
basic_ostream(basic_ostream&& os);
```

Effects: The state necessary to perform os.emit() is moved to *this.os.out=nullptr [Note: This disassociates os from its wrapped stream buffer ensuring destruction of os produces no output. —end note]

Postconditions: The value returned by this->rdbuf_wrapped() is the value returned by os.rdbuf_wrapped() prior to calling this constructor. Output stored in *os.rdbuf() prior to calling this constructor will be stored in *this->rdbuf() afterwards. os.rdbuf()->pbase()==os.rdbuf()->pptr() and nullptr == os.rdbuf_wrapped()

27.12.2.2 Destructor [synstream.ostream.dtor]

```
~basic_ostream();
```

Effects: Calls emit().

[Note: May destroy a mutex. —end note]

27.12.2.3 Assignment [synstream.ostream.assign]

```
basic_ostream& operator=(basic_ostream&& os);
```

Effects: Calls emit(). The state necessary to perform os.emit() is moved to *this.os.out=nullptr; [Note: This disassociates os from its wrapped stream buffer ensuring destruction of os produces no output. —end note]

Postcondition: nullptr == os.rdbuf_wrapped(). rdbuf_wrapped() returns the value previously returned by os.rdbuf_wrapped()

[Note: May destroy a mutex. —end note]

27.12.2.4 Member Functions [syncstream.ostream.mfun]

```
void emit();
```

Effects: Transfers the contents of the internal stream buffer to the stream buffer *out, so that they appear in the output stream as a contiguous sequence of characters. If and only if a flush was requested on *this, out->pubsync() is called on the stream buffer obtained in the constructor. A failure in transferring the characters or flushing calls, such as out having the value nullptr, result in setstate(ios::badbit).

Synchronization: All emit() calls transferring characters to the same stream buffer object appear to execute in a total order consistent with happens-before where each emit() call synchronizes-with subsequent emit() calls in that total order.

Remarks: May hold a lock uniquely associated with the underlying stream buffer object obtained in the constructor while transferring characters.

[Example: The function emit() can be used to catch exceptions from operations on the underlying stream.

```

    __osyncstream bout(cout);
    bout << "Hello, " << "World!" << endl;
    try {
        bout.emit();
    } catch ( ... ) {
        // stuff
    }
}

```

—end example]

streambuf type* rdbuf wrapped() const noexcept;

Returns: out

[Example: Obtaining the wrapped stream buffer with rdbuf_wrapped() allows wrapping it again with an osyncstream. For example,

```

{
    __osyncstream bout1(cout);
    bout1 << "Hello, ";
    {
        __osyncstream bout2(bout1.rdbuf_wrapped());
        bout2 << "Goodbye, " << "Planet!" << endl;
    }
    bout1 << "World!" << endl;
}

```

produces the *uninterleaved* output

```

Goodbye, Planet!
Hello, World!

```

—end example.]

Implementation

Two example implementations are available on github.com/PeterSommerlad/SC22WG21_Papers/workspace/Test_basic_osyncstream and https://github.com/PeterSommerlad/SC22WG21_Papers/tree/master/workspace/p0053_basic_osyncstreambuf.

The latter actually implements almost all of the mechanics in a stream buffer as proposed by Pablo Halpern where the first one is my original implementation with most of the mechanics stuck in the basic_osyncstream.

Revisions

This paper revises [P0053R1 C++ Synchronized Buffered Ostream](#)

- Provide a typedef for the wrapped stream buffer and use it to shorten the specification as suggested by Daniel Krügler.
- Provide move construction and move assignment and specify the moved-from state to be detached from the wrapped stream buffer.
- Rename get() to rdbuf_wrapped() and provide noexcept specification.
- Changed to explicitly rely on wrapping a stream buffer, instead of an ostream object and adjust explanations accordingly.

This paper revises [P0053R0 C++ Synchronized Buffered Ostream](#)

- Add remark to note that exchanging the stream buffer while the stream is wrapped causes undefined behavior and added a note to warn stream buffer implementers about the lock being held in emit(). Call setstate(badbit) if IO errors occur in emit().
- Replace code references to basic_streambuf by the term *stream buffer* introduced in [stream.buffer].
- Provide an example implementation.

- The lock is to be associate to the underlying `basic_streambuf` instead of the `basic_stream`.
- Added an Allocator constructor parameter.
- Moves destructor example to `emit()`.
- Clarifies wording about synchronization and flushing (several times).
- List the new header in corresponding table.
- Provide type aliases in `<iosfwd>`.
- Removed copy constructor in favor of providing `get()`.
- Notify that move construction and assignment is deleted.
- Moved class `noteflush_streambuf` into an implementation note.
- Add a design subsection that states that a header test is a sufficient feature test.

This paper revises [N4187 C++ Ostream Buffers](#)

- Updated introduction with recent history.
- Rename `ostream_buffer` to `osyncstream` to reflect its appearance is more like a stream than like a buffer.
- Add an example of using `osyncstream` as a temporary object.
- Add an example of a `osyncstream` constructed with another `osyncstream`.
- Clarify the behavior of nested `osyncstream` executions.
- Clarify the behavior of exceptions occuring with the `osyncstream` destructor.
- Clarify the deferral of flush from the `osyncstream`'s `streambuf` to the final `basic_ostream`.
- Limit the number of references to `noteflush_stringbuf` in anticipation of the committee removing it from the specification.
- Rename `noteflush_stringbuf` to `noteflush_streambuf` to hide possible implementation details.
- Change the base class of `noteflush_streambuf` from `basic_stringbuf` to `basic_streambuf`.

[N4187](#) revised [N4069 C++ Ostream Buffers](#)

- Added note to sync as suggested by BSI via email.

[N4069](#) revised [N3978 C++ Ostream Buffers](#)

- Added a Design section.
- Clarify the reference capturing behavior of the `ostream_buffer` constructors.
- Added `noexcept` and `const` as appropriate to members.
- Added note on throwing wrapped streams.
- Change the `noteflush_stringbuf` public member variable `needsflush` to a public member query function `flushed`.
- Removed the public member function `noteflush_stringbuf::clear`.
- Minor synopsis formatting changes.
- Incorporated feedback from SG1 and Dietmar Kühl in specific in Rapperswil.

[N3978](#) revised [N3892 C++ Ostream Buffers](#)

- Flush the `ostream` if and only if the `ostream_buffer` was flushed.
- Add the `clear_for_reuse` function.
- Change the design from inheriting from `basic_ostream` to using a `noteflush_stringbuf`, which is a slightly modified `basic_stringbuf`. The modification is to note the flush rather than act upon it.

[N3892](#) revised [N3750 C++ Ostream Buffers](#)

- Change name to `basic_ostream_buffer` and add the usual typedefs.
- Change interface to inherit from `basic_ostringstream` rather than provide access to a member of that type.
- Add a Revisions section.

References

[PSL]

The Open Group Base Specifications Issue 6, IEEE Std 1003.1, 2004 Edition, functions, flockfile,
<http://pubs.opengroup.org/onlinepubs/009695399/functions/flockfile.html>